

PROJECT REPORT ON

WAZUH-VT-THREATATTACK

An Integrated Framework for Automated
Threat Intelligence Incident Response

ABSTRACT

This project focuses on the deployment and configuration of Wazuh, an open-source Security Information and Event Management (SIEM) platform, within a Kali Linux environment. The system is integrated with multiple threat intelligence and monitoring tools, including VirusTotal, AlienVault OTX, MITRE ATT&CK, File Integrity Monitoring (FIM), and Active Response. The primary objective of this project is to develop a centralized security monitoring environment capable of real-time threat detection, analysis, and response.

A single-node Wazuh environment was successfully deployed using Docker Engine on Kali Linux and configured through the command line. The setup includes monitoring of an agent system for security events. VirusTotal integration enables automated file hash analysis for malware detection, while AlienVault OTX provides threat intelligence to identify malicious IP addresses. MITRE ATT&CK mapping helps in understanding attacker behavior by linking events to known tactics and techniques. File Integrity Monitoring ensures system integrity by detecting unauthorized changes, and Active Response automates mitigation actions.

The system was tested using simulated threats such as EICAR test files and malicious IP lookups. The results confirm that the integrations work effectively, improving detection accuracy and enabling faster response to security incidents.

This project demonstrates how Wazuh, combined with external threat intelligence and automation, can provide an efficient and scalable solution for real-time threat detection and response.

TABLE OF CONTENTS

1. INTRODUCTION	4
1.1 WAZUH SIEM OVERVIEW	4
1.2 KEY FEATURES & INTEGRATIONS	4
2. PROJECT OBJECTIVES	6
3. SYSTEM PREREQUISITES	7
4. IMPLEMENTATION METHODOLOGY	8
4.1 ENVIRONMENT PREPARATION	8
4.2 WAZUH STACK DEPLOYMENT USING DOCKER.....	10
4.3 WAZUH AGENT DEPLOYMENT ENDPOINT MONITORING ...	13
4.4 FILE INTEGRITY MONITORING FIM TEST	17
4.5 VIRUSTOTAL THREAT INTELLIGENCE INTEGRATION	19
4.6 ALIENVault OTX THREAT INTELLIGENCE INTEGRATION	23
4.7 MITRE ATT&CK INTEGRATION	28
4.8 ACTIVE RESPONSE IMPLEMENTATION	31
5. THREAT VALIDATION & RESULTS	36
6. CONCLUSION	37

INTRODUCTION

1.1 WAZUH SIEM OVERVIEW

Wazuh is a comprehensive, open-source security monitoring platform used for threat detection, integrity monitoring, and incident response. In this project, Wazuh serves as the central SIEM (Security Information and Event Management) and XDR (Extended Detection and Response) hub, providing visibility across the entire monitored infrastructure.

1.2 KEY FEATURES & INTEGRATIONS

▼ File Integrity Monitoring (FIM)

The FIM module in Wazuh is engineered to track changes in system files, directories, and Windows registry keys in real-time. By calculating and comparing cryptographic checksums (MD5, SHA1, and SHA256), Wazuh identifies unauthorized modifications, deletions, or new file creations. This is a foundational security control used to detect the presence of rootkits, unauthorized configuration changes, and malware persistence in sensitive system areas.

▼ VirusTotal API Integration

To bridge the gap between detection and threat intelligence, this project integrates the VirusTotal API. When a new file is detected or an existing one is modified (monitored by FIM), Wazuh automatically extracts the file hash and queries VirusTotal's massive database of over 70 antivirus engines. This

provides an automated layer of malware analysis, ensuring that known malicious binaries are flagged instantly without manual intervention.

▼ AlienVault OTX (Open Threat Exchange)

While VirusTotal focuses on files, AlienVault OTX provides global intelligence on network-based threats. By integrating the OTX API, the Wazuh manager can cross-reference logs (such as firewall or web server logs) against millions of Indicators of Compromise (IOCs). If a monitored agent interacts with a known malicious IP address, domain, or URL, an high-priority alert is generated, allowing for immediate containment of potential C2 (Command and Control) communication.

▼ MITRE ATT&CK Framework Mapping

Every alert generated by the Wazuh engine is automatically mapped to the MITRE ATT&CK framework. This mapping translates technical logs into adversary behavior. For example, a suspicious PowerShell command isn't just a log entry; it is identified by its Technique ID (e.g., T1059.001). This provides security analysts with the context needed to understand the "tactics" (why the attacker is doing it) and the "techniques" (how they are doing it).

▼ Automated Active Response

The Active Response module transforms Wazuh from a passive monitoring tool into an active defense system. Based on specific alert triggers—such as a brute-force attack or a confirmed malware detection—Wazuh can execute pre-defined scripts on the endpoint. These actions include dropping malicious firewall connections, terminating unauthorized processes, or isolating a compromised host from the network, significantly reducing the "Mean Time to Respond" (MTTR).

PROJECT OBJECTIVES

The primary goal of this project is to architect and validate a proactive security monitoring framework. The specific objectives include:

- To deploy a Wazuh SIEM environment using Docker Engine on Kali Linux for monitoring and analyzing security events.
- To install and configure Wazuh agents for endpoint monitoring and real-time data collection.
- To establish secure communication between the Wazuh manager and agents to ensure reliable event reporting.
- To integrate VirusTotal for detecting malicious files using file hash analysis.
- To integrate AlienVault OTX for obtaining threat intelligence and identifying suspicious IP addresses and indicators.
- To enable MITRE ATT&CK framework integration to classify detected threats based on attacker tactics and techniques.
- To implement File Integrity Monitoring (FIM) and Active Response to detect unauthorized system changes and automatically respond to security incidents.
- To test and verify the system for proper communication, accurate detection, and effective alert generation across all integrations.

SYSTEM PREREQUISITES

System Requirements:

Single-node stack deployment:

- Operating system: Linux or Windows
- Architecture: AMD64
- CPU At least 4 cores
- Memory: At least 8 GB of RAM for the Docker host
- Disk space: At least 50 GB storage for Docker images and data

volumes

Wazuh agent deployment:

- Operating system: Linux or Windows
- Architecture: AMD64
- CPU At least 2 cores
- Memory: At least 1 GB of RAM for the Docker host
- Disk space: At least 10 GB storage for Docker images and logs

IMPLEMENTATION METHODOLOGY

Required Software :

Install Docker Desktop (Windows) or Docker Engine (Linux)

Here we used Linux version.

4.1 ENVIRONMENT PREPARATION

Before deploying the SIEM stack, the host system was updated and the necessary container orchestration tools were installed via the terminal.

- Step 1 System Update

```
sudo apt update && sudo apt upgrade -y
```

- Step 2 Docker Engine Installation

Instead of a GUI-based Docker Desktop, the project utilizes the Docker Engine and Docker Compose plugin for better resource management on Kali Linux.

```
sudo apt install docker.io -y
```

- Step 3 Service Management

Enabled and started the Docker daemon to ensure it runs automatically on system boot.

```
sudo systemctl start docker  
sudo systemctl enable docker
```

- Step 4 Verify Docker Service Status

Before proceeding, it was essential to confirm that the Docker daemon was initialized and running correctly.

```
sudo systemctl status docker
```



```
(shifana@kali)~  
$ systemctl status docker  
  
● docker.service - Docker Application Container Engine  
   Loaded: loaded (/usr/lib/systemd/system/docker.service; enabled; preset: enabled)  
   Active: active (running) since Sun 2026-03-29 17:23:45 IST; 5 days ago  
 Invocation: d5bc935619934a6bb467c8f59ac7708b  
TriggeredBy: ● docker.socket  
   Docs: https://docs.docker.com  
  Main PID: 1254 (dockerd)  
    Tasks: 119  
  Memory: 77M (peak: 1.4G, swap: 30.9M, swap peak: 47.1M)  
    CPU: 18min 13.682s  
   CGroup: /system.slice/docker.service  
           └─ 1254 /usr/sbin/dockerd -H fd:// --containerd=/run/containerd/containerd.sock  
              └─ 538104 /usr/sbin/docker-proxy -proto udp -host-ip 0.0.0.0 -host-port 514 -container-ip 172.18.0.2 -container-port 514  
                 └─ 538111 /usr/sbin/docker-proxy -proto udp -host-ip :: -host-port 514 -container-ip 172.18.0.2 -container-port 514  
                    └─ 538114 /usr/sbin/docker-proxy -proto tcp -host-ip 0.0.0.0 -host-port 9200 -container-ip 172.18.0.3 -container-port 9200  
                       └─ 538120 /usr/sbin/docker-proxy -proto tcp -host-ip 0.0.0.0 -host-port 1514 -container-ip 172.18.0.2 -container-port 1514  
                          └─ 538126 /usr/sbin/docker-proxy -proto tcp -host-ip :: -host-port 9200 -container-ip 172.18.0.3 -container-port 9200  
                             └─ 538139 /usr/sbin/docker-proxy -proto tcp -host-ip :: -host-port 1514 -container-ip 172.18.0.2 -container-port 1514  
                                └─ 538171 /usr/sbin/docker-proxy -proto tcp -host-ip 0.0.0.0 -host-port 1515 -container-ip 172.18.0.2 -container-port 1515  
                                   └─ 538191 /usr/sbin/docker-proxy -proto tcp -host-ip :: -host-port 1515 -container-ip 172.18.0.2 -container-port 1515  
                                      └─ 538203 /usr/sbin/docker-proxy -proto tcp -host-ip 0.0.0.0 -host-port 55000 -container-ip 172.18.0.2 -container-port 55000  
                                         └─ 538210 /usr/sbin/docker-proxy -proto tcp -host-ip :: -host-port 55000 -container-ip 172.18.0.2 -container-port 55000  
                                            └─ 538409 /usr/sbin/docker-proxy -proto tcp -host-ip 0.0.0.0 -host-port 443 -container-ip 172.18.0.4 -container-port 5601  
                                               └─ 538416 /usr/sbin/docker-proxy -proto tcp -host-ip :: -host-port 443 -container-ip 172.18.0.4 -container-port 5601
```

Note: Press Q on your keyboard to exit the status view and return to the command line.)

- Step 5 Post-Installation Configuration

To avoid using `sudo` for every command, the current user was added to the Docker group.

```
sudo usermod -aG docker $USER  
# A system reboot was performed here to apply changes  
reboot
```

- Step 6 Installation Verification

Confirmed that the engine was active and responding correctly.

```
docker --version  
docker run hello-world
```

```

(shifana@kali)-[~]
$ docker --version
Docker version 27.5.1+dfsg4, build cab968b3

(shifana@kali)-[~]
$ docker run hello-world

Hello from Docker!
This message shows that your installation appears to be working correctly.

To generate this message, Docker took the following steps:
 1. The Docker client contacted the Docker daemon.
 2. The Docker daemon pulled the "hello-world" image from the Docker Hub.
    (amd64)
 3. The Docker daemon created a new container from that image which runs the
    executable that produces the output you are currently reading.
 4. The Docker daemon streamed that output to the Docker client, which sent it
    to your terminal.

To try something more ambitious, you can run an Ubuntu container with:
$ docker run -it ubuntu bash

Share images, automate workflows, and more with a free Docker ID:
https://hub.docker.com/

For more examples and ideas, visit:
https://docs.docker.com/get-started/

```

- Step 7 Install Docker Compose

Installed the orchestration tool required to manage the multi-container Wazuh stack.

```

sudo apt install docker-compose -y
docker-compose --version

```

4.2 WAZUH STACK DEPLOYMENT USING DOCKER

- Step 1 Version Control & Repository Acquisition

```

# Install Git and verify the version
sudo apt install git -y
git --version

```

```
# Clone the official Wazuh Docker repository
git clone https://github.com/wazuh/wazuh-docker.git -b v4.1
4.0
```

```
(shifana@kali)~$ git clone https://github.com/wazuh/wazuh-docker.git -b v4.10.0
Cloning into 'wazuh-docker'...
remote: Enumerating objects: 16814, done.
remote: Counting objects: 100% (561/561), done.
remote: Compressing objects: 100% (120/120), done.
remote: Total 16814 (delta 505), reused 442 (delta 441), pack-reused 16253 (from 3)
Receiving objects: 100% (16814/16814), 7.20 MiB | 423.00 KiB/s, done.
Resolving deltas: 100% (9178/9178), done.
Note: switching to '2f3f00fc518ff3023260132f38e77989e8699fe2'.

You are in 'detached HEAD' state. You can look around, make experimental
changes and commit them, and you can discard any commits you make in this
state without impacting any branches by switching back to a branch.

If you want to create a new branch to retain commits you create, you may
do so (now or later) by using -c with the switch command. Example:

    git switch -c <new-branch-name>

Or undo this operation with:

    git switch -

Turn off this advice by setting config variable advice.detachedHead to false
```

- Navigate to the single-node directory to execute all the following commands.

```
cd wazuh-docker/single-node
```

- Step 2 Generating SSL Certificates

Following the industry standard for encrypted communication, SSL certificates were generated for the Indexer, Dashboard, and Server.

```
# Run the certificate generator container
docker compose -f generate-indexer-certs.yml run --rm gener
ator
```

```

(shifana@kali)-[~/wazuh-docker/single-node]
$ docker compose -f generate-indexer-certs.yml run --rm generator
[+] Creating 1/1
✓ Network single-node_default Created
[+] Running 5/5
✓ generator Pulled
✓ 17d0386c2fff Pull complete
✓ 7ce91ec7d1d3 Pull complete
✓ 5249716d429c Pull complete
✓ d7003467fd14 Pull complete
The tool to create the certificates exists in the in Packages bucket
29/03/2026 12:22:44 INFO: Generating the root certificate.
29/03/2026 12:22:45 INFO: Generating Admin certificates.
29/03/2026 12:22:46 INFO: Admin certificates created.
29/03/2026 12:22:46 INFO: Generating Wazuh indexer certificates.
29/03/2026 12:22:46 INFO: Wazuh indexer certificates created.
29/03/2026 12:22:47 INFO: Generating Filebeat certificates.
29/03/2026 12:22:47 INFO: Wazuh Filebeat certificates created.
29/03/2026 12:22:47 INFO: Generating Wazuh dashboard certificates.
29/03/2026 12:22:48 INFO: Wazuh dashboard certificates created.
Moving created certificates to the destination directory
Changing certificate permissions
Setting UID indexer and dashboard
Setting UID for wazuh manager and worker

```

- The generated certificates will be stored in the `wazuhdocker/singlenode/config/wazuh_indexer_ssl_certs` directory.

```

admin-key.pem
admin.pem
root-ca.key
root-ca-manager.key
root-ca-manager.pem
root-ca.pem
wazuh.dashboard-key.pem
wazuh.dashboard.pem
wazuh.indexer-key.pem
wazuh.indexer.pem
wazuh.manager-key.pem
wazuh.manager.pem

```

- Step 3 Starting Wazuh Containers

Start the Wazuh Docker deployment using the docker compose command:

```
docker-compose up -d
```

```

shifana@kali:~/wazuh-docker/single-node
docker-compose up -d
Running 15/46
wazuh.indexer [ ] Pulling
c3f933517985 Waiting 45.8s
d1a552c49659 Waiting 42.8s
63a161db8e7 Waiting 41.9s
98243c7f9b6d Waiting 41.9s
f27e691e0145 Waiting 41.9s
b9599a87e562 Waiting 41.9s
a9a0f905a798 Waiting 41.9s
e6cbb8b4a877 Waiting 41.9s
340009edd041 Waiting 41.9s
ca04432f6c89 Waiting 41.9s
335f1418e3f Waiting 41.9s
542c6d74e55a Waiting 41.9s
d4a4b8629191 Waiting 41.9s
wazuh.manager [ ] Pulling
07a4be1b2929 Pull complete 45.7s
0e36c91e306c Downloading [=====] 42.89MB/54.63MB 19.6s
8ec1385289f9 Download complete 11.7s
eb815531b448 Download complete 15.2s
b07618b2c67d Download complete 17.6s
28f8a3a32483 Downloading [=====] 30.78MB/168MB 41.9s
2281d1d18e2c Waiting 41.9s
c78ede8aed79 Waiting 41.9s
290f6dd4c8c8 Waiting 41.9s
f6093835602c Waiting 41.9s
87d4facf9818 Waiting 41.9s
03b3e75a0b0d Waiting 42.8s
51c3b3aacc81 Waiting 42.8s
ec85b0421587 Waiting 42.8s
d5493c4b5489 Waiting 42.8s
c8078ce27266 Waiting 42.8s
9ba662af5090 Waiting 42.8s
7ead693346e2 Waiting 42.8s
wazuh.dashboard [ ] Pulling 177MB / 296.8MB Pulling 45.8s
d4c3c94e5e10 Pull complete 19.5s
32d7aa3de5b7 Pull complete 19.6s
faa5573b8321 Pull complete 19.7s
0e057b9b51c8 Pull complete 19.7s
f19a301c2281 Pull complete 19.8s
c809b33b78a Pull complete 19.8s
cb8f2ab322f8 Pull complete 19.9s

```

Note: The initial deployment may take 5 10 minutes as Docker pulls the necessary images for the Manager, Indexer, Dashboard, and Filebeat.

- Step 4 Service Verification

Once the deployment process was completed, the status of the individual containers was verified to ensure all services were "Up" and healthy.

```
# List all active containers
docker ps
```

```

shifana@kali:~/wazuh-docker/single-node
$ docker ps
CONTAINER ID   IMAGE                                COMMAND                  CREATED        STATUS        PORTS
991e87691ded   wazuh/wazuh-dashboard:4.14.0       /entrypoint.sh*         11 minutes ago Up 25 seconds 443/tcp, 0.0.0.0:443->5601/tcp, [::]:443->5601/tcp
afa7ca8297ef   wazuh/wazuh-indexer:4.14.0         /entrypoint.sh open.*    11 minutes ago Up 27 seconds 0.0.0.0:9200->9200/tcp, :::9200->9200/tcp
cb05bed759c2   wazuh/wazuh-manager:4.14.0         /init*                  11 minutes ago Up 11 minutes 0.0.0.0:1514-1515->1514-1515/tcp, :::1514-1515->1514-1515/tcp, 0.0.0.0:514->514/udp, :::514->514/udp, 0.0.0.0:55000->55000/tcp, :::55000->55000/tcp, 1516/tcp
single-node-wazuh.manager-1

```

4.3 WAZUH AGENT DEPLOYMENT END POINT MONITORING

To enable real-time monitoring and log collection, the Wazuh Agent was installed on the Kali Linux system. This allows the system to send security events and file integrity data to the Wazuh Manager for analysis.

- Step 1 Repository Configuration and Installation

The Wazuh repository was added to the system, and the agent package was installed using the following commands:

```
# Import Wazuh GPG Key curl -s https://packages.wazuh.com/key/GPG-KEY-WAZUH | gpg --no-default-keyring --keyring gnupg-ring:/usr/share/keyrings/wazuh.gpg --import

# Set permissions
chmod 644 /usr/share/keyrings/wazuh.gpg

# Add Wazuh repository
echo "deb [signed-by=/usr/share/keyrings/wazuh.gpg] https://packages.wazuh.com/4.x/apt/ stable main" | tee -a /etc/apt/sources.list.d/wazuh.list

# Update packages
sudo apt update

# Install Wazuh agent
sudo apt install wazuh-agent -y
```

- Step 2 Agent Configuration

After installing the Wazuh Agent, the configuration file must be edited to connect the agent with the Wazuh Manager.

Configuration File Path:

`/var/ossec/etc/ossec.conf`

Open the configuration file using a text editor:

```
sudo nano /var/ossec/etc/ossec.conf
```

Locate the <server> section and modify the manager address to:
<address>127.0.0.1 /address>


```
<ossec_config>
  <client>
    <server>
      <address>127.0.0.1</address>
      <port>1514</port>
      <protocol>tcp</protocol>
    </server>
    <config-profile>kali, kali2026, kali2026.1</config-profile>
    <notify_time>20</notify_time>
    <time-reconnect>60</time-reconnect>
    <auto_restart>yes</auto_restart>
    <crypto_method>aes</crypto_method>
  </client>
```

Here, 127.0.0.1 refers to localhost, which means the agent and manager are running on the same system.

After making the changes, save the file and exit the editor.

- Step 3 Service Activation

The agent service was enabled and started using the following commands:

```
sudo systemctl daemon-reload
sudo systemctl enable wazuh-agent
sudo systemctl start wazuh-agent
```

This ensured that the agent runs automatically and continuously monitors the system.

- Step 4 Accessing the Wazuh dashboard

After deploying the single-node stack, you can access the Wazuh dashboard using your Docker host's IP address or localhost.

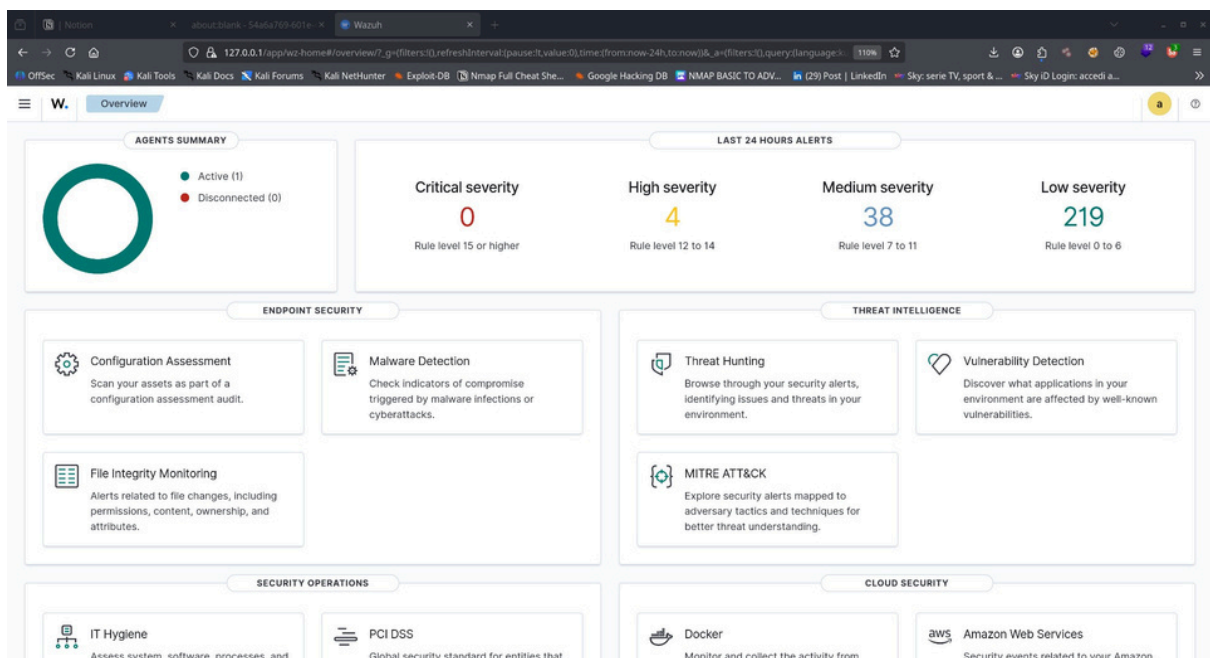
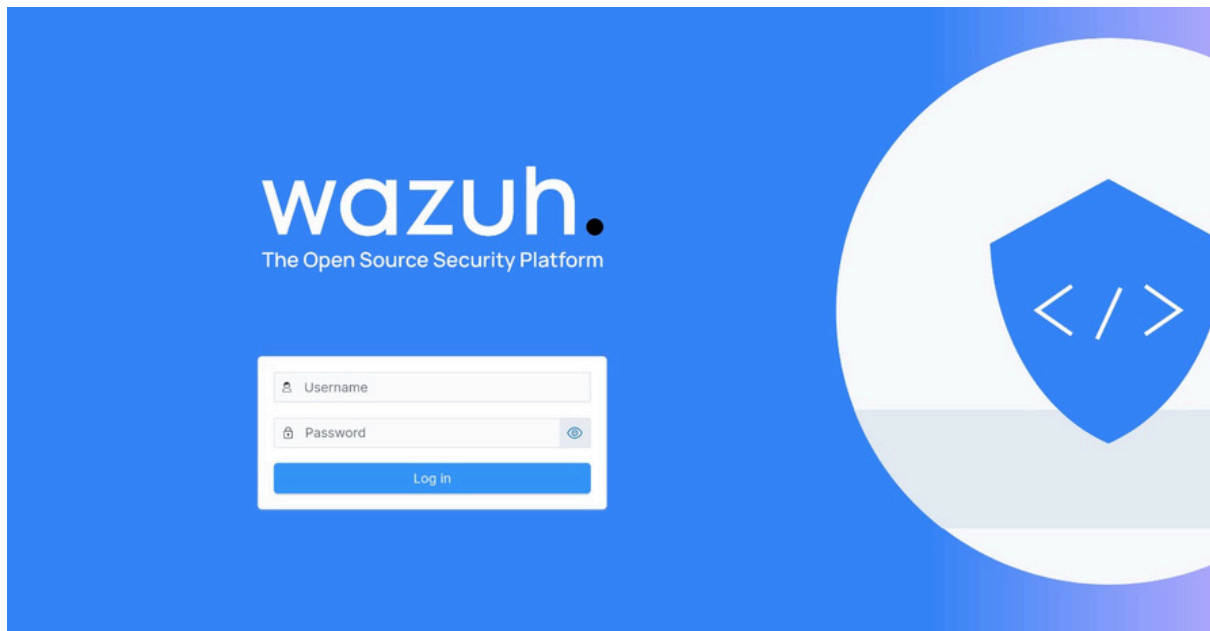
The dashboard was opened in a web browser using the following URL

<http://127.0.0.1>

Default login credentials:

Username: admin

Password: SecretPassword



4.4 FILE INTEGRITY MONITORING FIM TEST

File Integrity Monitoring (FIM) is used to detect unauthorized changes in system files and directories. It helps identify potential security issues such as file modifications, deletions, or suspicious activities.

Step 1 Edit the Configuration File

To enable FIM, the Wazuh Agent configuration file must be modified.

Configuration File Path:

`/var/ossec/etc/ossec.conf`

Open the file using a text editor:

```
sudo nano /var/ossec/etc/ossec.conf
```

Inside the file, locate the `<syscheck>` section under the File integrity monitoring and add the following line:

```
<directories check_all="yes" whodata="yes">/home/kali/Downloads</directories>
```

```
<!-- File integrity monitoring -->
<syscheck>
  <disabled>no</disabled>

  <!-- Frequency that syscheck is executed default every 12 hours -->
  <frequency>43200</frequency>

  <scan_on_start>yes</scan_on_start>

  <!-- Directories to check (perform all possible verifications) -->
  <directories>/etc,/usr/bin,/usr/sbin</directories>
  <directories>/bin,/sbin,/boot</directories>

  <directories check_all="yes" whodata="yes">/home/shifana/Downloads</directories>

  <!-- Files/directories to ignore -->
  <ignore>/etc/mtab</ignore>
  <ignore>/etc/hosts.deny</ignore>
  <ignore>/etc/mail/statistics</ignore>
  <ignore>/etc/random-seed</ignore>
  <ignore>/etc/random.seed</ignore>
  <ignore>/etc/adjtime</ignore>
  <ignore>/etc/httpd/logs</ignore>
  <ignore>/etc/utmpx</ignore>
  <ignore>/etc/wtmpx</ignore>
  <ignore>/etc/cups/certs</ignore>
  <ignore>/etc/dumpdates</ignore>
  <ignore>/etc/svc/volatile</ignore>
```

After adding the configuration, save the file and exit the editor.

What does this do?

- `check_all="yes"` → monitors all attributes and computes hashes
- `whodata="yes"` → logs user, process, time
- Works in real-time mode

- Step 2 Restart the Wazuh Agent

After modifying the configuration, restart the Wazuh Agent to apply the changes:

```
sudo systemctl restart wazuh-agent
```

- Step 3 Trigger a File Change

To test FIM, make a change inside the monitored directory:

- Create a new file
- Modify an existing file
- Delete a file

These actions will trigger alerts in Wazuh.

- Step 4 Verify Alerts in Dashboard

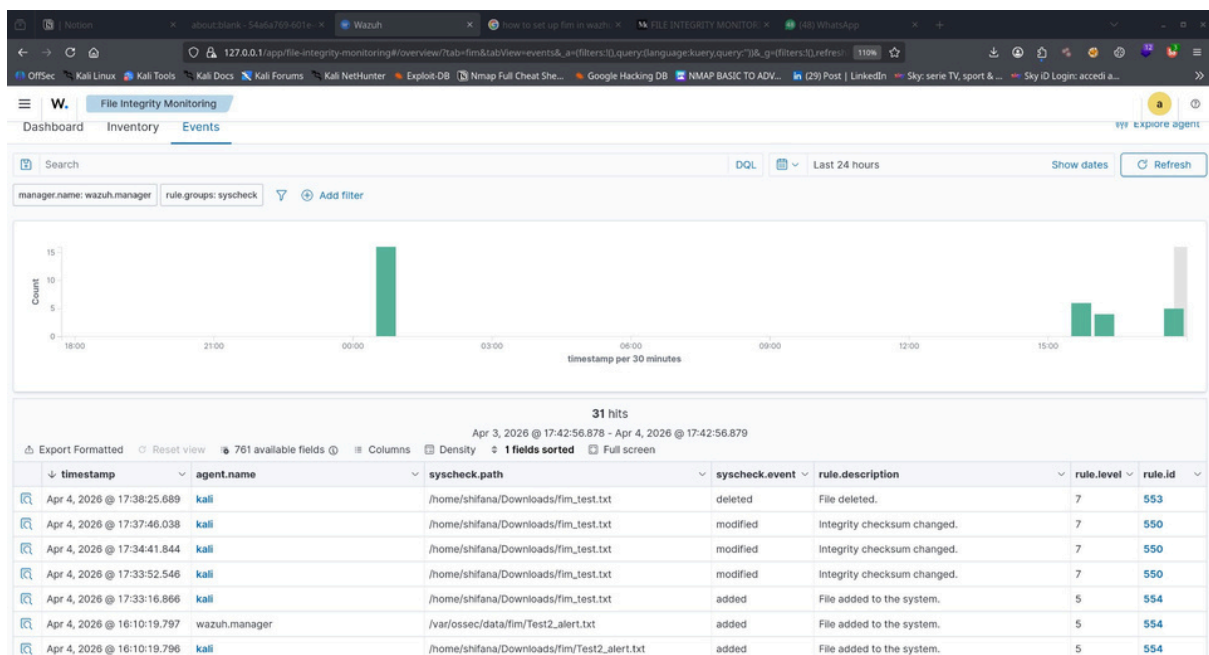
Login to the Wazuh Dashboard and navigate

Go to File Integrity Monitoring → Events

You will observe alerts related to file changes such as:

- File added
- File modified
- File deleted

This confirms that FIM is working correctly.



4.5 VIRUSTOTAL THREAT INTELLIGENCE INTEGRATION

This section explains the integration of VirusTotal with Wazuh to enable automated malware detection using file hash analysis.

- Step 1 Obtaining VirusTotal API Key
 - a) Login to VirusTotal account from <https://www.virustotal.com/gui/home/upload>
 - b) Obtain your VirusTotal API Key
- Step 2 Preparing Directory and Permissions

A dedicated directory was created on the Kali Linux host system to store monitored files. Proper permissions were assigned to ensure that Docker containers can access this directory.

```
sudo mkdir -p /home/kali/Downloads/fim
sudo chmod -R 755 /home/kali/Downloads/fim
sudo chown -R $USER:$USER /home/kali/Downloads/fim
```

This step ensures that the directory is accessible for both reading and writing operations.

- Step 3 Configuring Docker Volume Mapping

To allow Wazuh to monitor files from the host system, the local directory was mapped to the Wazuh container directory.

Open the Docker Compose file:

```
nano ~/wazuh-docker/single-node/docker-compose.yml
```

Under the volumes section, add the following line:

```
/home/kali/Downloads/fim:/var/ossec/data/fim
```

```
volumes:
  - wazuh_api_configuration:/var/ossec/api/configuration
  - wazuh_etc:/var/ossec/etc
  - wazuh_logs:/var/ossec/logs
  - wazuh_queue:/var/ossec/queue
  - wazuh_var_multigroups:/var/ossec/var/multigroups
  - wazuh_integrations:/var/ossec/integrations
  - wazuh_active_response:/var/ossec/active-response/bin
  - wazuh_agentless:/var/ossec/agentless
  - wazuh_wodles:/var/ossec/wodles
  - filebeat_etc:/etc/filebeat
  - filebeat_var:/var/lib/filebeat
  - ./config/wazuh_indexer_ssl_certs/root-ca-manager.pem:/etc/ssl/root-ca.pem
  - ./config/wazuh_indexer_ssl_certs/wazuh.manager.pem:/etc/ssl/filebeat.pem
  - ./config/wazuh_indexer_ssl_certs/wazuh.manager-key.pem:/etc/ssl/filebeat.key
  - ./config/wazuh_cluster/wazuh_manager.conf:/wazuh-config-mount/etc/ossec.conf
  - /home/shifana/Downloads/fim:/var/ossec/data/fim
```

This step links the local Kali directory to the Wazuh container so that files can be accessed and monitored inside the container. Without this mapping, the container cannot access host files, and integrations such as VirusTotal will not function properly.

- Step 4 Editing Wazuh Manager Configuration

The Wazuh Manager configuration file was modified to enable monitoring and integrate VirusTotal.

File Path:

~/wazuh-docker/single-node/config/wazuh_cluster/wazuh_manager.conf

Open the file:

```
nano ~/wazuh-docker/single-node/config/wazuh_cluster/wazuh_
manager.conf
```

a) Enable Syscheck Monitoring

Locate the <syscheck> section and configure it as follows:

```
<syscheck>
<disabled>no</disabled>
<directories check_all="yes" realtime="yes">/var/ossec/da
ta/fim</directories>
</syscheck>
```

```
<!-- File integrity monitoring -->
<syscheck>
  <disabled>no</disabled>

  <!-- Frequency that syscheck is executed default every 12 hours -->
  <frequency>43200</frequency>

  <scan_on_start>yes</scan_on_start>

  <!-- Generate alert when new file detected -->
  <alert_new_files>yes</alert_new_files>

  <!-- Don't ignore files that change more than 'frequency' times -->
  <auto_ignore frequency="10" timeframe="3600">no</auto_ignore>

  <!-- Directories to check (perform all possible verifications) -->
  <directories>/etc,/usr/bin,/usr/sbin</directories>
  <directories>/bin,/sbin,/boot</directories>

  <directories check_all="yes" realtime="yes">/var/ossec/data/fim</directories>
```

b) Add VirusTotal Integration Block

Scroll to the end of the file and add the following block before </ossec_config>:

```
<integration>
<name>virustotal</name>
```

```
<api_key>YOUR_API_KEY_HERE</api_key>
<group>syscheck</group>
<alert_format>json</alert_format>
</integration>
```

Save the file and exit.

```
<integration>
<name>virustotal</name>
<api_key>efea2e85ad732def8b62f3a4ed424ca1a241726b381ac17cbeb0b2b6b57c368b</api_key>
<group>syscheck</group>
<alert_format>json</alert_format>
</integration>
</ossec_config>
```

- Step 5 Applying Changes

Since configuration files were modified, the Docker containers were restarted to apply changes

Go to `cd ~/wazuh-docker/single-node`

```
docker-compose down
docker-compose up -d
```

This ensures that all updated configurations are loaded correctly.

- Step 5 Testing the Integration

To test the setup, a safe test file (EICAR) was created inside the monitored directory.

```
echo 'X5O!P%@AP[4\PZX54(P^)7CC)7}$EICAR-STANDARD-ANTIVIRUS-
TEST-FILE!$H+H*' > /home/shifana/Downloads/fim/Test1_alert.
txt
```

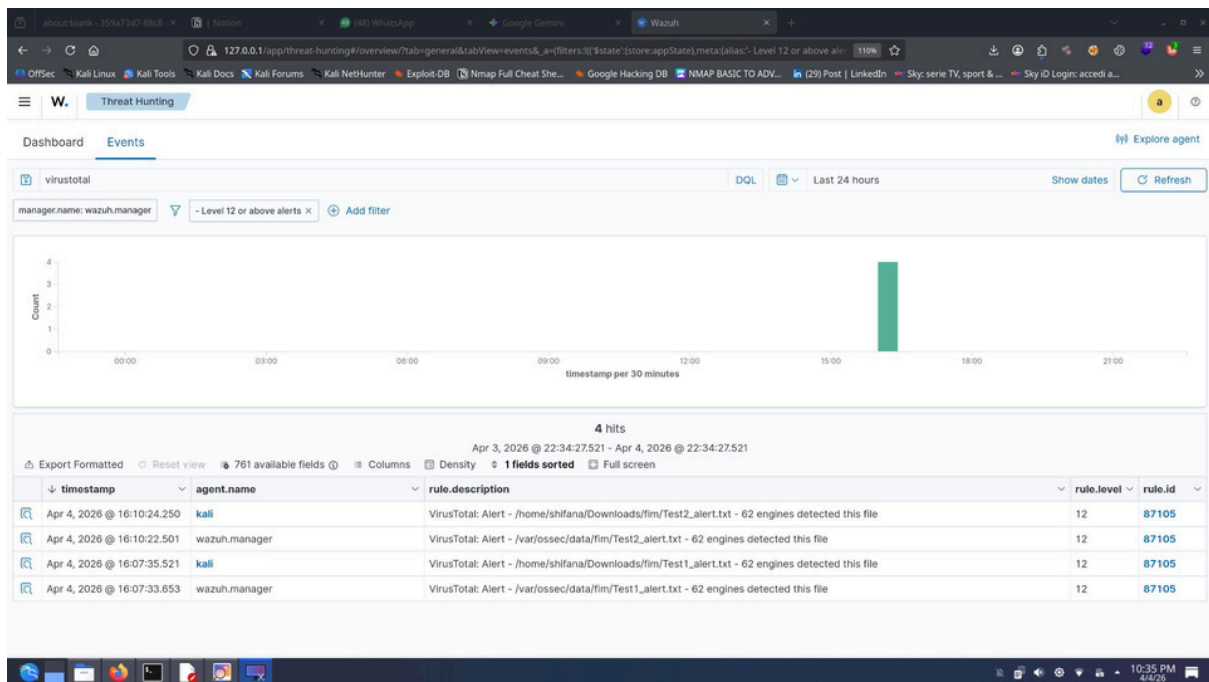
This file simulates a malicious file and triggers VirusTotal analysis without causing any harm.

```
(shifana@kali) - [~/wazuh-docker/single-node]
$ echo 'X50!P%AP[4\PZX54(P^)7CC)7]$EICAR-STANDARD-ANTIVIRUS-TEST-FILE!$H+H*' > /home/shifana/Downloads/fim/Test1_alert.txt
```

- Step 7 Verification of Results

a) Check Wazuh Dashboard

Go to Wazuh Dashboard Threat Hunting → Events



4.6 ALIENVAULT OTX THREAT INTELLIGENCE INTEGRATION

AlienVault OTX provides real-time, community-driven threat intelligence. Integrating it with Wazuh enables automatic reputation checks for suspicious IP addresses and domains, enhancing threat detection and incident analysis.

- Step 1 Obtain OTX API Key
Loginto:<https://otx.alienvault.com>

Go to Profile → API Key

Copy your API key

- Step 2 Create Custom Integration Script

Access the Wazuh Manager container and create the script

```
docker exec -it single-node-wazuh.manager-1 bin/bash
```

- Step 3 Create Integration Script

Navigate to the integrations directory and create the script file.

```
cd /var/ossec/integrations/  
nano custom-alienvault.py
```

```
#!/usr/bin/env python3  
import sys  
import json  
import requests  
from datetime import datetime  
  
# Your API Key  
OTX_API_KEY = "edf5a34c1472ed0ea5c1e1d978ada29ed4fcf9a347a6  
cadd53dc2e13c0eed240"  
OTX_URL = "https://otx.alienvault.com/api/v1/indicators/IPv  
4/{}/general"  
  
def main():  
    # Read the alert  
    if len(sys.argv) > 1:  
        with open(sys.argv[1]) as f:  
            alert = json.load(f)  
    else:  
        alert = json.load(sys.stdin)
```



```

#Extract IP
ip=alert.get("data", {}).get("srcip")
if not ip:
    return

# Get current time for the "PDF look"
now = datetime.now().strftime("%Y-%m-%d %H:%M:%S.%f")[:-3]

headers = {"X-OTX-API-KEY": OTX_API_KEY}
response = requests.get(OTX_URL.format(ip), headers=headers)

if response.status_code == 200:
    data = response.json()
    pulses = data.get("pulse_info", {}).get("count", 0)

    #THIS PRINTS THE EXACT FORMAT FROM YOUR PDF PHOTO
    print(f"{now} - Querying AlienVault OTX for {ip}") print(f"
    {now} - AlienVault: {ip} found in {pulses}
    pulses.")

if __name__ == "__main__":
    main()

```

Save the file and exit.



```
single-node: docker — Konsole
single-node: zsh × single-node: zsh × single-node: docker ×
GNU nano 8.1 /var/ossec/integrations/custom-alienvault.py
#!/usr/bin/env python3
import sys
import json
import requests
from datetime import datetime # Added this for the timestamp

# The API key is added here so the script can run by itself in the terminal
OTX_API_KEY = "edf5a34c1472ed0ea5c1e1d978ada29ed4fcf9a347a6cadd53dc2e13c0eed240"
OTX_URL = "https://otx.alienvault.com/api/v1/indicators/IPV4/general"

def main():
    if len(sys.argv) > 1:
        with open(sys.argv[1]) as f:
            alert = json.load(f)
    else:
        alert = json.load(sys.stdin)

    ip = alert.get("data", {}).get("srcip")
    if not ip:
        return

    # Create the timestamp to match your PDF guide
    now = datetime.now().strftime("%Y-%m-%d %H:%M:%S.%f")[:-3]
    headers = {"X-OTX-API-KEY": OTX_API_KEY}
    response = requests.get(OTX_URL.format(ip), headers=headers)

    if response.status_code == 200:
        data = response.json()
        pulses = data.get("pulse_info", {}).get("count", 0)

        # UPDATED: This prints the exact date/time and text format from your PDF
        print(f"[{now}] - Querying AlienVault OTX for {ip}")
        print(f"[{now}] - AlienVault: {ip} found in {pulses} pulses.")

if __name__ == "__main__":
    main()
```

This script reads alert data, extracts the source IP address, and queries the AlienVault OTX API to retrieve threat intelligence information.

- Step 6 Configure Integration in Wazuh

Edit the Wazuh configuration file:

nano /var/ossec/etc/ossec.conf

Add the following integration block:

```
<integration>
<name>custom-alienvault</name>
<api_key>YOUR_API_KEY_HERE</api_key>
<level>3</level>
<alert_format>json</alert_format>
</integration>
```

Save the file and exit.

```
<integration>
<name>custom-alienvault</name>
<api_key>edf5a34c1472ed0ea5c1e1d978ada29ed4fcf9a347a6cadd53dc2e13c0eed240</api_key>
<level>3</level>
<alert_format>json</alert_format>
</integration>
</ossec_config>
```

- Step 7 Restart Wazuh Manager

Apply the changes by restarting the Wazuh Manager service.

```
/var/ossec/bin/wazuh-control restart
```

This ensures that the new integration settings are loaded.

- Step 8 Test the Integration

Create a test alert file to simulate a suspicious IP event.

```
# Construct a simulated Wazuh alert with metadata and source IP
cat > /tmp/test-alert.json <<EOF
{
  "rule": { "id": "100001", "level": 10, "description": "OTX Test Alert" },
  "agent": { "id": "001", "name": "kali-agent" },
  "data": { "srcip": "198.51.100.18" },
  "full_log": "Suspicious connection detected from 198.51.100.18"
}
EOF

# Manually trigger the OTX reputation check
/var/ossec/integrations/custom-alienvault.py /tmp/test-alert.json
```

- Step 9 Verify Results

Check the terminal output for threat intelligence details such as the number of pulses associated with the IP address.

```

(shifana@kali)-[~/wazuh-docker/single-node]
$ docker exec -it single-node-wazuh.manager-1 /bin/bash
bash-5.2# cat > /tmp/test-alert.json <<EOF
{
  "rule": { "id": "100001", "level": 10, "description": "OTX Test Alert" },
  "agent": { "id": "001", "name": "kali-agent" },
  "data": { "srcip": "198.51.100.18" },
  "full_log": "Suspicious connection detected from 198.51.100.18"
}
EOF
bash-5.2# /var/ossec/integrations/custom-alienvault.py /tmp/test-alert.json
2026-04-05 05:31:07.454 - Querying AlienVault OTX for 198.51.100.18
2026-04-05 05:31:07.454 - AlienVault: 198.51.100.18 found in 0 pulses.
bash-5.2# █

```

4.7 MITRE ATT&CK INTEGRATION

This section explains how MITRE ATT&CK mapping was implemented in Wazuh to classify detected activities based on attacker techniques and behaviors.

- Step 1 Install Auditd (Logging System)

Auditd is a Linux auditing tool that helps Wazuh monitor low-level system activities.

Install Auditd using the following command:

```
sudo apt update && sudo apt install auditd -y
```

This ensures that system-level events are recorded for analysis.

- Step 2 Add Audit Rule for Monitoring

To track sensitive file access (like password files), an audit rule was added. Open the audit rules file:

```
sudo nano /etc/audit/rules.d/audit.rules
```

Add the following line at the bottom:

```
-w /etc/shadow -p wa -k shadow_access
```

This rule monitors changes and access to the `/etc/shadow` file.

```
## First rule - delete all
-D

## Increase the buffers to survive stress events.
## Make this bigger for busy systems
-b 8192

## This determine how long to wait in burst of events
--backlog_wait_time 60000

## Set failure mode to syslog
-f 1
-w /etc/shadow -p wa -k shadow_access
```

Reload the rules:

```
sudo augenrules --load
```

Reload the rules:

- Step 3 Configure Wazuh Agent to Read Audit Logs

The Wazuh Agent must be configured to collect audit logs.

Open the configuration file:

```
sudo nano /var/ossec/etc/ossec.conf
```

Add the following block before the closing `</ossec_config>` tag:

```
<localfile>
<log_format>audit</log_format>
<location>/var/log/audit/audit.log</location>
</localfile>
```

This step ensures that audit logs are sent to the Wazuh Manager.

```
<localfile>  
  <log_format>nmapg</log_format>  
  <location>/var/log/nmap_scan.xml</location>  
</localfile>
```

- Step 4 Restart Wazuh Agent

Apply the configuration changes by restarting the agent:

```
sudo systemctl restart wazuh-agent
```

- Step 5 Simulate Attack – Privilege Escalation

Run the following command:

```
sudo cat /etc/shadow
```

MITRE Technique: Privilege Escalation T1548.003

Purpose: Attempt to access sensitive password hashes.

- Step 6 Simulate Attack – Persistence

Run the following command:

```
sudo useradd -o -u 0 -g 0 backdoored_user
```

MITRE Technique: Persistence T1136.001

Purpose: Creating a hidden root-level user.

- Step 7 Simulate Attack – Brute Force Attempt

Run the following command:

su fakeuser

Enter an incorrect password multiple times.

MITRE Technique: T1110 Brute Force)

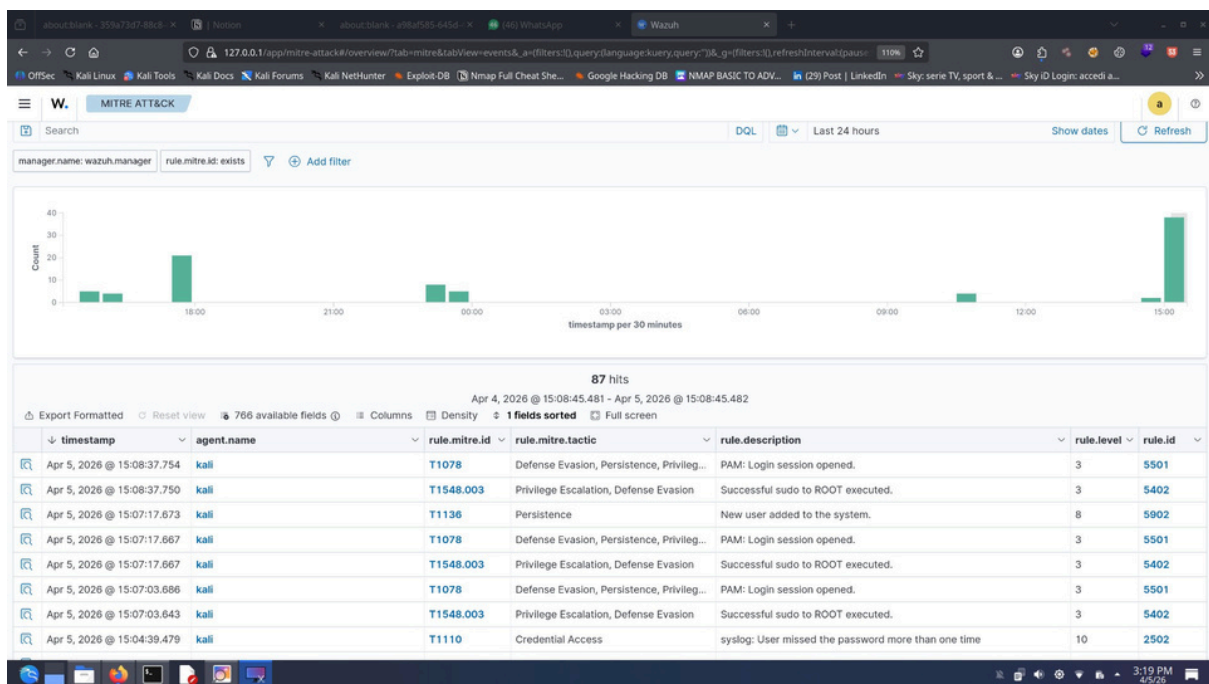
Purpose: Simulates repeated login attempts to gain unauthorized access

- Step 8 Verify Results in Wazuh Dashboard

Login to the Wazuh Dashboard and navigate to:

MITRE ATT&CK → Overview

Filter by Agent Name or Technique ID



4.8 ACTIVE RESPONSE IMPLEMENTATION

This section explains how Active Response was configured in Wazuh to automatically respond to detected threats. In this project, the response was

triggered based on VirusTotal alerts, allowing the system to take action when a malicious file is detected.

- Step 1 Configure Active Response in Wazuh Manager

The Wazuh Manager configuration file was edited to enable automatic response.

Open the configuration file:

```
nano ~/wazuh-docker/single-node/config/wazuh_cluster/wazuh_
manager.conf
```

Add the following block in active response area:

```
<active-response>
<command>firewall-drop</command>
<location>local</location>
<rules_id>87105</rules_id>
<timeout>600</timeout>      </active-
response>
```

This configuration triggers the firewall-drop action whenever a VirusTotal alert (Rule ID 87105) is detected. The action blocks the malicious source for 600 seconds.

```
<active-response>
  <command>firewall-drop</command>
  <location>local</location>
  <rules_id>87105</rules_id>
  <timeout>600</timeout>
</active-response>
```

- Step 2 Define the Command

Ensure that the firewall-drop command is properly defined.

Add the following block in the same configuration file if not already present:


```
<command>
<name>firewall-drop</name>
<executable>firewall-drop</executable>
<expect>srcip</expect>
<timeout_allowed>yes</timeout_allowed>
</command>
```

This defines the script that will be executed when the active response is triggered.

```
<command>
  <name>firewall-drop</name>
  <executable>firewall-drop</executable>
  <expect>srcip</expect>
  <timeout_allowed>yes</timeout_allowed>
</command>
```

- Step 3 Grant Docker Network Permissions

Since Wazuh is running inside Docker, additional permissions are required to allow it to modify firewall rules.

Open the Docker Compose file:

```
nano ~/wazuh-docker/single-node/docker-compose.yml
```

Under the `wazuh.manager` section, add:

```
cap_add:
- NET_ADMIN
- NET_RAW
```

These permissions allow the container to control network-level firewall rules.

```
GNU nano 8.7.1 docker-compose.yml
# Wazuh App Copyright (C) 2017, Wazuh Inc. (License GPLv2)
services:
  wazuh-manager:
    image: wazuh/wazuh-manager:4.14.0
    hostname: wazuh.manager
    restart: always
    cap_add:
      - NET_ADMIN
      - NET_RAW
    ulimits:
      memlock:
        soft: -1
        hard: -1
      nofile:
        soft: 65536
        hard: 65536
    ports:
      - "1514:1514"
      - "1515:1515"
      - "514:514/udp"
      - "5555:5555"
```

- Step 4 Apply Changes and Restart Containers

To apply all configuration changes, restart the Wazuh Docker environment:

```
cd ~/wazuh-docker/single-node
docker-compose down
docker-compose up -d
```

This ensures that the updated configuration and permissions are loaded.

- Step 5 Trigger Malicious Event

To test Active Response, generate a threat using VirusTotal detection.

Create a test malware file (EICAR test file):

```
echo 'X5O!P%#@AP[4\PZX54(P^)7CC)7}$EICAR-STANDARD-ANTIVIRUS-
TEST-FILE!$H+H*' > /home/shifana/Downloads/fim/test_virus.txt
```

This file will trigger a VirusTotal alert.

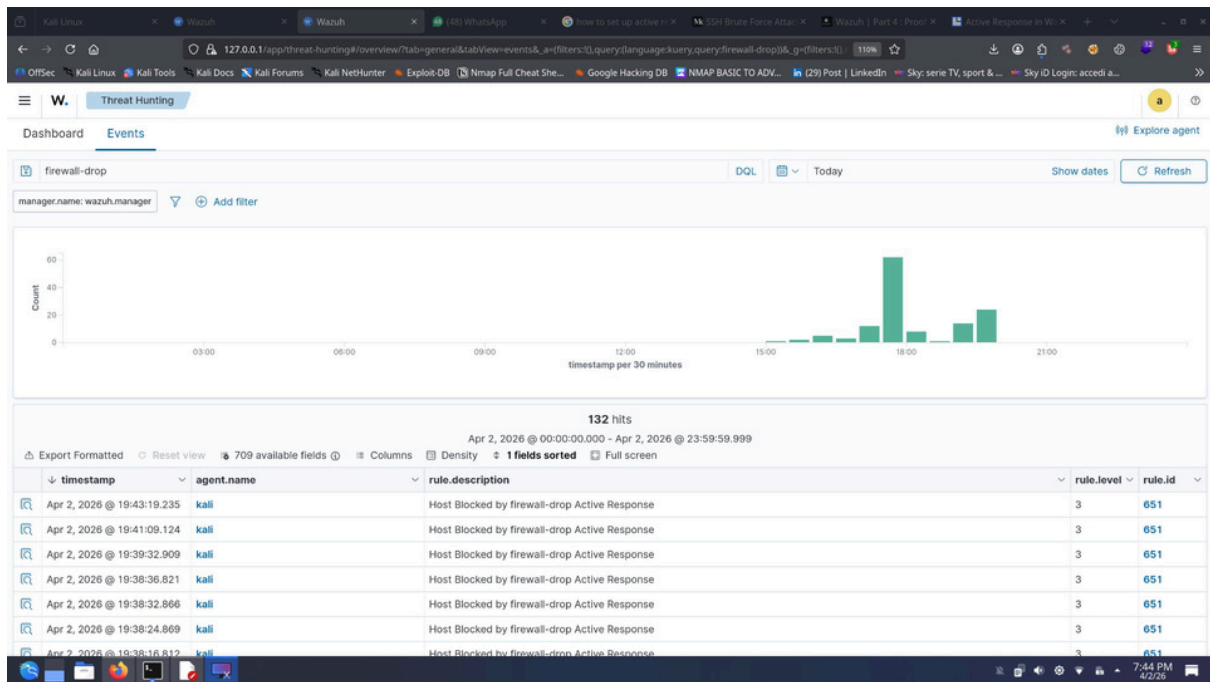
You can also Run a malware file or trigger a VirusTotal alert.

Check logs: `tail -f /var/ossec/logs/active-responses.log`

- Step 7 View Result in Dashboard

Go to Wazuh Dashboard and navigate to:

Threat Hunting → Events



THREAT VALIDATION AND RESULTS

This section presents the practical validation of the Wazuh SIEM environment by simulating real-world attack scenarios and analyzing the system's response. Various security mechanisms configured in the project, including File Integrity Monitoring FIM , VirusTotal integration, AlienVault OTX threat intelligence, MITRE ATT&CK mapping, and Active Response, were thoroughly tested. During testing, multiple events such as file modifications, malware simulation using the EICAR test file, suspicious IP analysis, and unauthorized system activities were generated. The Wazuh platform successfully detected these activities and generated alerts in real time. Each alert was analyzed through the Wazuh Dashboard, providing detailed insights into the nature and severity of the detected threats.

The VirusTotal integration verified malicious files using hash-based analysis, while the AlienVault OTX integration enriched alerts with external threat intelligence data. The MITRE ATT&CK framework mapped detected activities to standardized attacker techniques, improving the understanding of attack patterns. Additionally, the Active Response mechanism automatically executed predefined actions such as blocking malicious sources, demonstrating the system's ability to respond to threats without manual intervention.

The results confirm that the configured environment is capable of accurate threat detection, efficient alert generation, and automated response, thereby validating the effectiveness of the implemented security solution.

CONCLUSION

This project successfully demonstrated the deployment and configuration of a comprehensive Wazuh SIEM environment on Kali Linux using Docker. By integrating multiple security components, the system was able to provide centralized monitoring, real-time threat detection, and automated incident response.

The implementation of File Integrity Monitoring ensured continuous tracking of critical system files, while VirusTotal and AlienVault OTX integrations enhanced the system's ability to identify and analyze potential threats using external intelligence sources. The MITRE ATT&CK integration provided a structured approach to understanding attacker behavior, and the Active Response feature enabled immediate action against detected threats.

Overall, the project highlights the importance of combining multiple security layers to build a robust and proactive cybersecurity framework. It demonstrates how Wazuh can be effectively used as a scalable and efficient SIEM solution for modern security operations.

This implementation not only strengthens the security posture of the system but also provides valuable practical experience in security monitoring, threat analysis, and incident response, making it highly relevant for real-world cybersecurity applications.